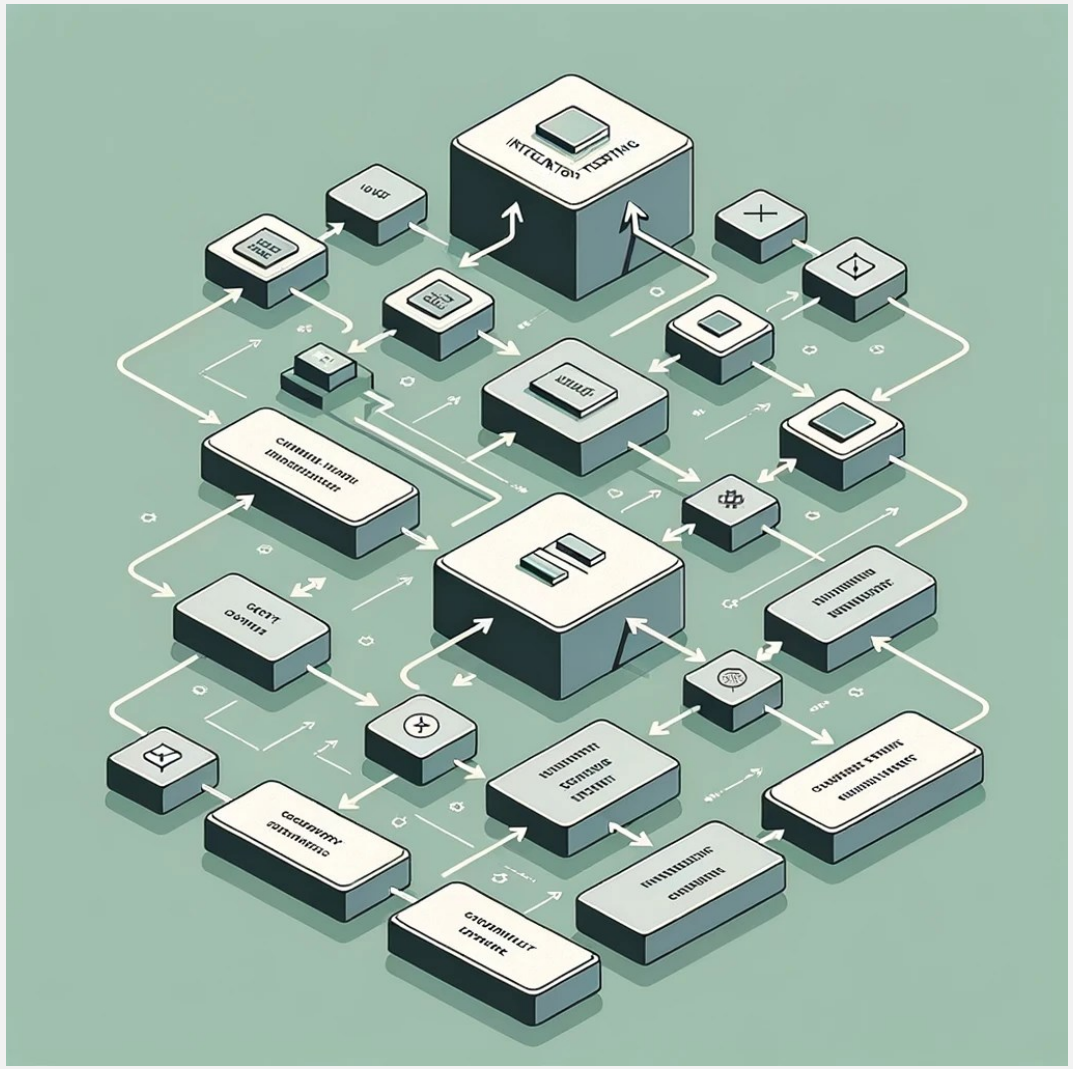
Manchester Metropolitan University

# Week 4

# Integration Testing

Dr Lewis Evans

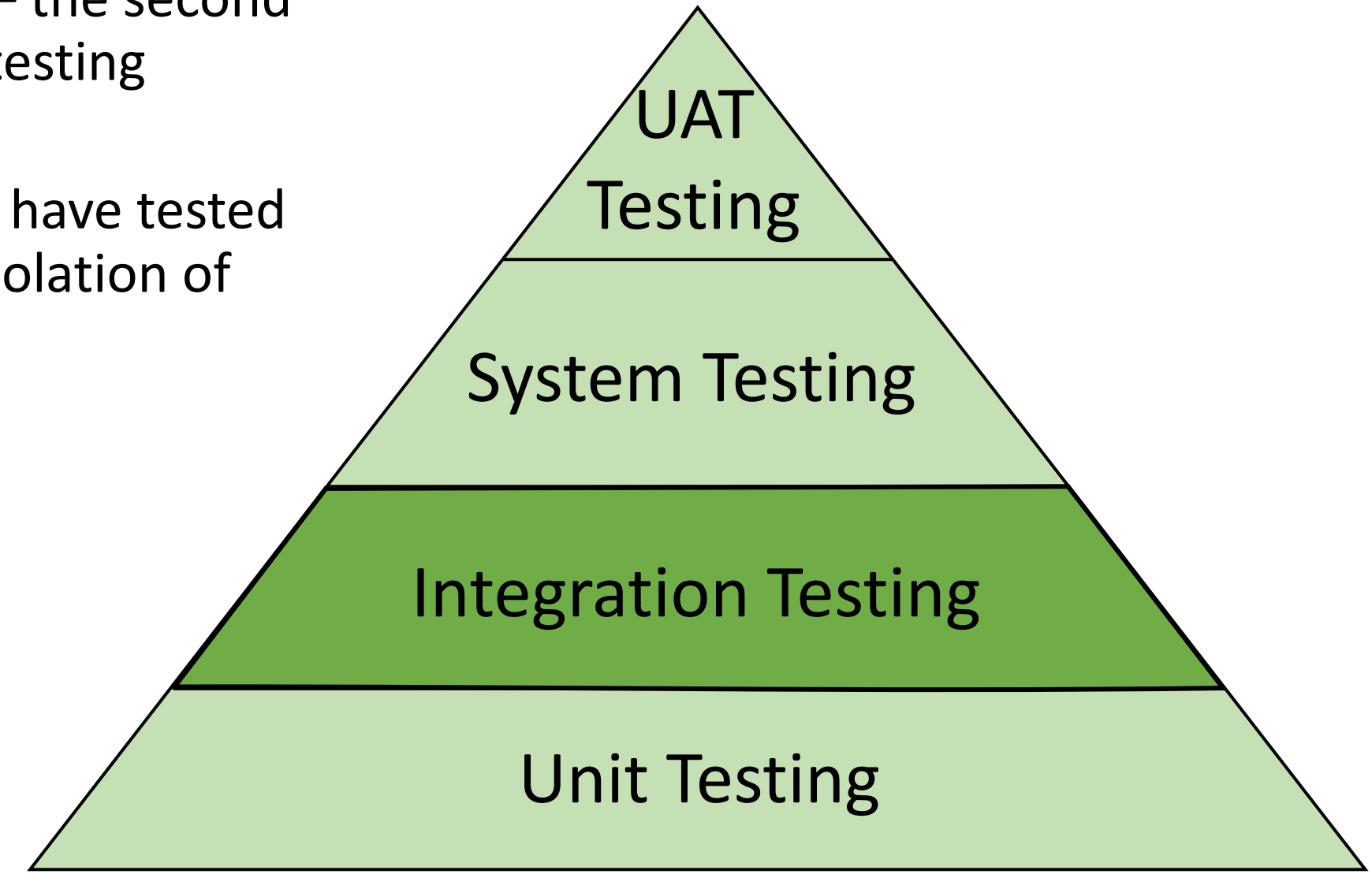


# OUTLINE

- Integration Testing
  - Definition and importance
  - Types of integration testing
  - Differences between unit testing and integration testing
  - Examples of integration testing

# INTEGRATION TESTING

- Integration Testing – the second testing level in our testing pyramid
- Performed after we have tested individual units in isolation of each other



# INTEGRATION TESTING

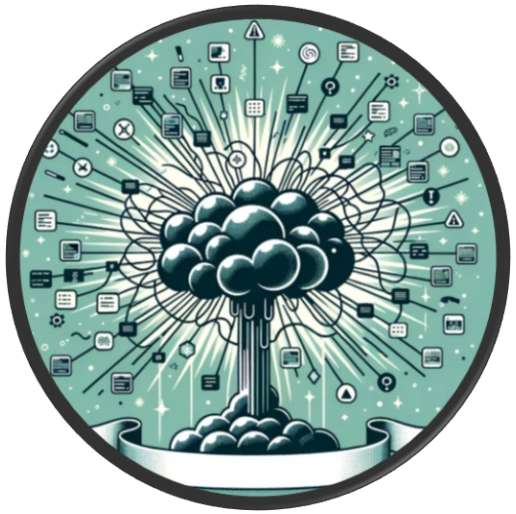
- **Evaluates Interoperability** – Ensures that different units or components can operate together effectively.
- **Beyond Unit Testing** – Validates the communication and operation between multiple integrated components.
- **Mimics Real-world Scenarios** – Tests under conditions similar to actual operating environments.
- **Data Integrity** – Checks the consistency and integrity of data as it moves between units or systems.

# WHAT CAN WE PERFORM INTEGRATION TESTING ON?

- We can test the integration between many different components, and as one of the test principles (seen in week 2) states: “Testing is context-dependent”
- **E.g. Flight Reservation System**
  - **Search and Results**: Test if the search function correctly displays available flights.
  - **Login and Dashboard**: Check if logging in successfully directs to the user’s personalised dashboard.
  - **Add to Cart**: Verify that selecting a flight actually adds it to the user's cart.
  - **Booking Confirmation**: Ensure the system sends a confirmation message after successful booking.
  - **Cancellation and Refunds**: Test if cancelling a booking triggers a refund process.
  - **Frequent Flyers**: Confirm that frequent flyer miles are correctly updated after each flight.

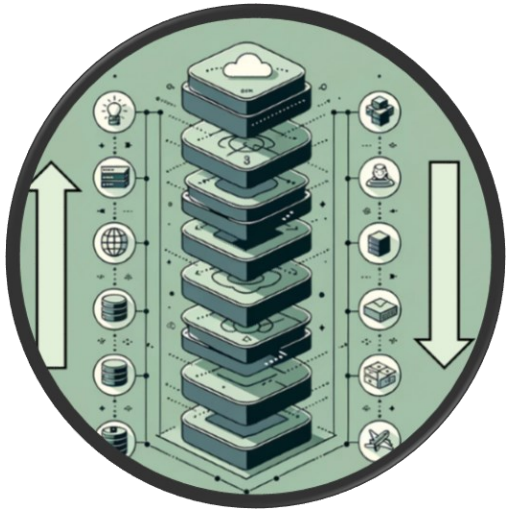
# TYPES OF INTEGRATION TESTING

- Although integration testing is concerned with testing multiple components and how they interact with each other, there are different ways of achieving this

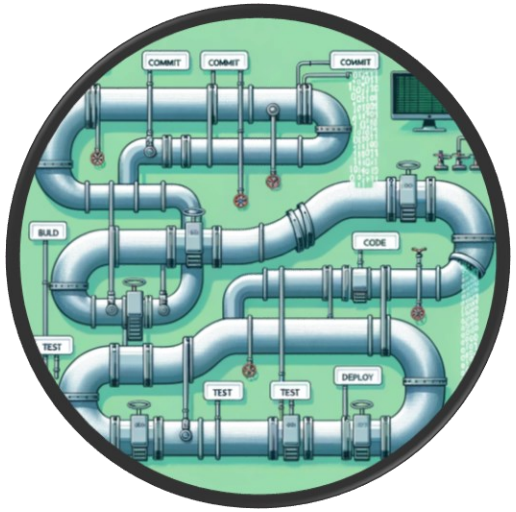


***Big Bang***

***Incremental (e.g. Top-down, Bottom-up)***



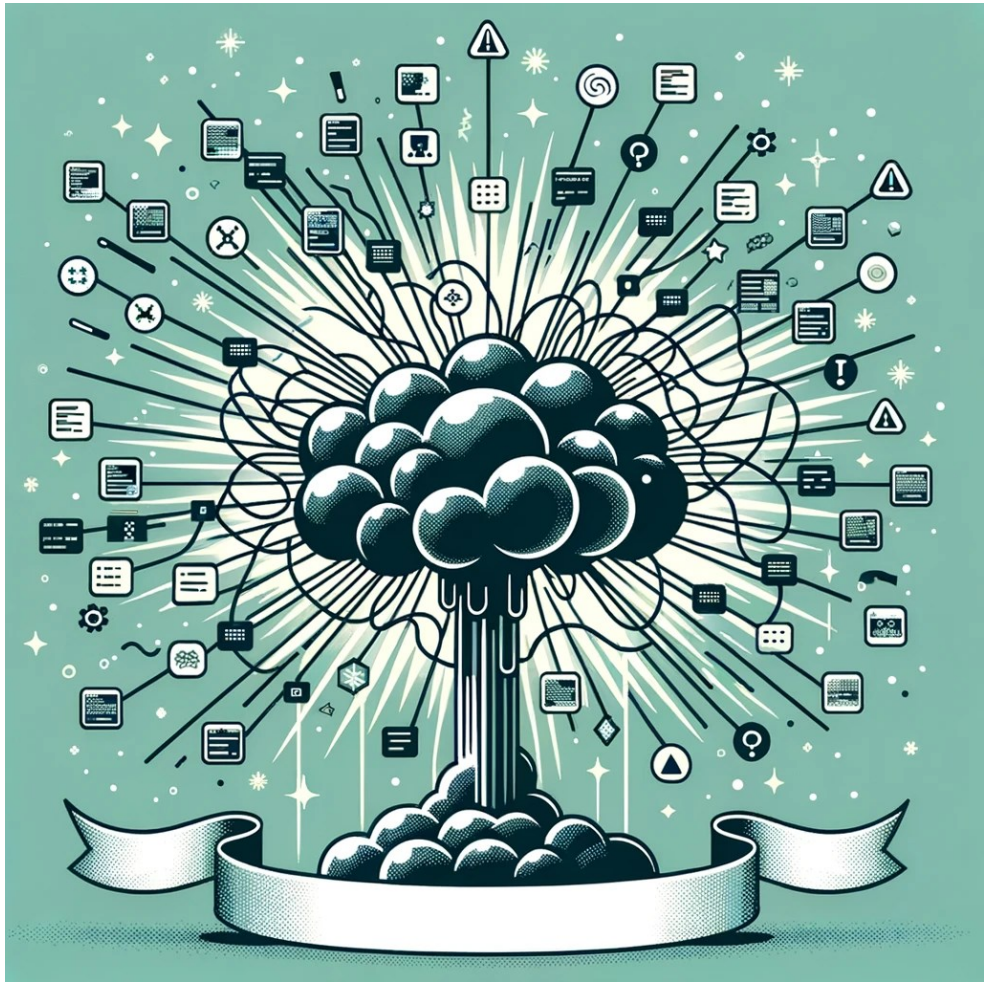
***Sandwich (Hybrid)***




***Continuous Integration***

# BIG BANG

- **Big Bang:** Testing after all modules are integrated
- **When to Use**: Smaller projects, quick validations
- **Pros and Cons**
  - **Pros**: Easier setup, immediate system-wide testing
  - **Cons**: Difficult to isolate defects, not suitable for large projects

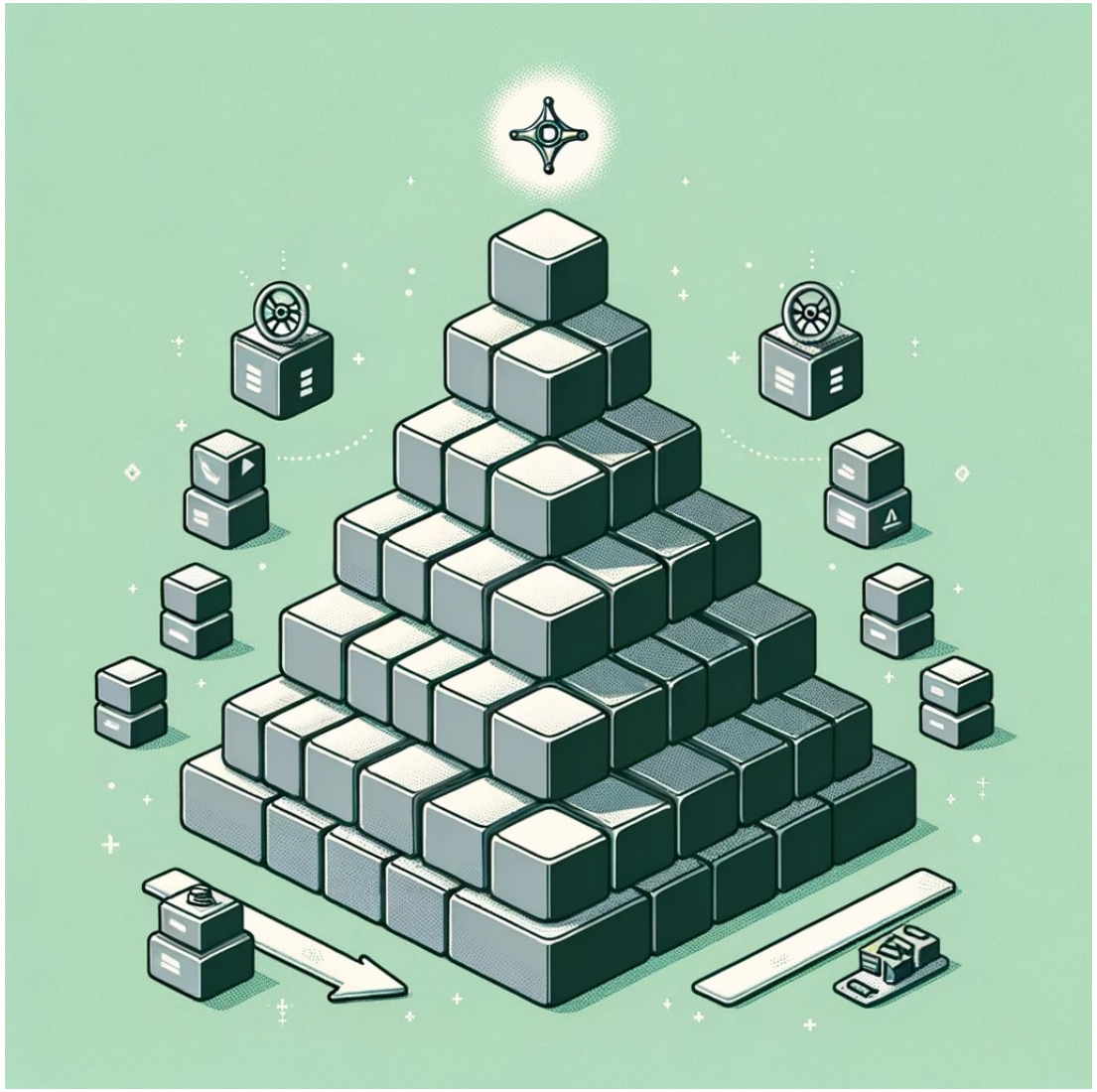


# INCREMENTAL: TOP-DOWN

- Testing from top module to bottom, using **stubs** for lower modules
  - **Stub**: a simple implementation of a software component (often a function or an object) that mimics the behaviour of the component with which it interacts but doesn't implement any real functionality
- **When to Use:** When high-level logic is developed first
- **Pros and Cons**
  - **Pros**: Early discovery of high-level issues, easier debugging
  - **Cons**: Requires stubs, may overlook lower-level issues

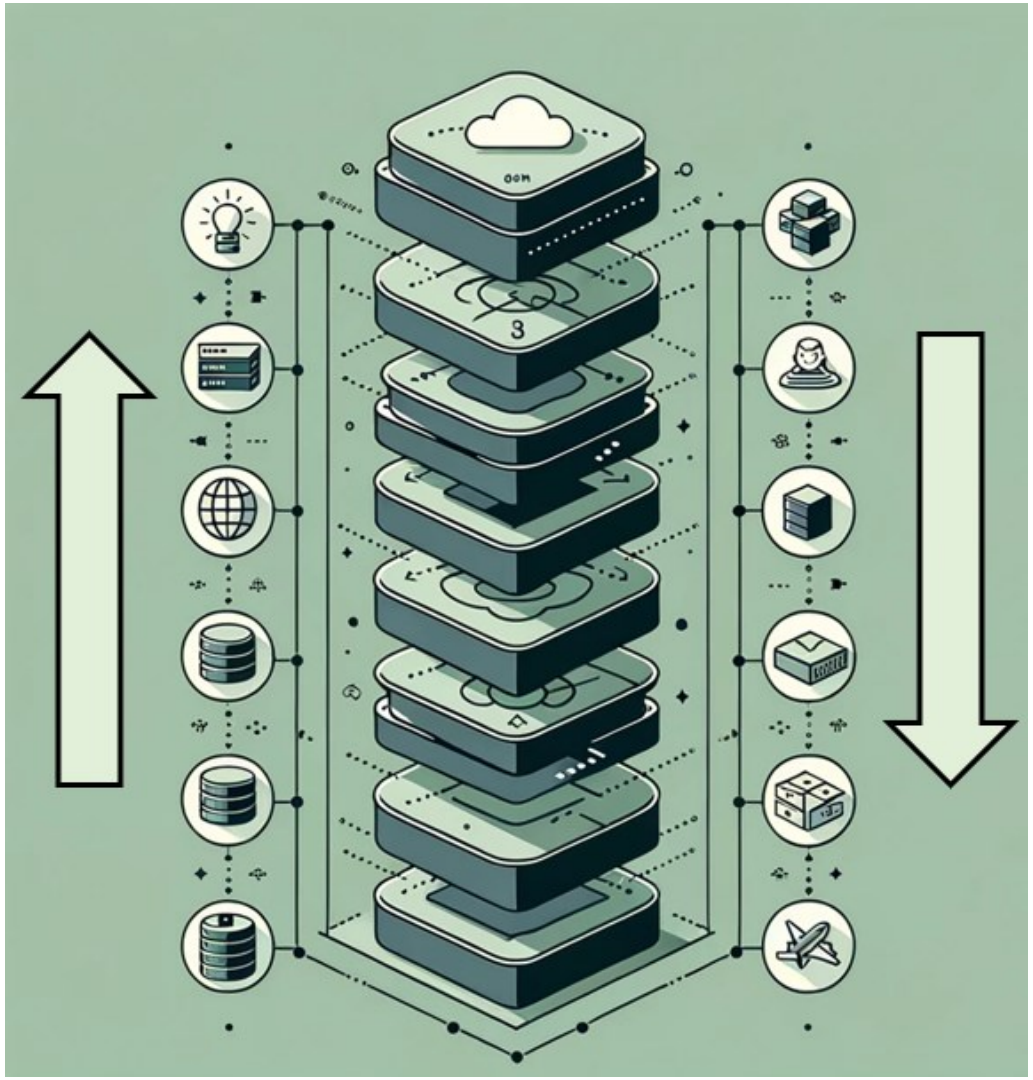
# INCREMENTAL: BOTTOM-UP

- Testing from lower to upper modules, using **drivers** for higher modules
  - **Driver**: component used to initiate and control the execution of a lower-level component, which is yet to be integrated into the system.
- **When to Use**: When low-level components are ready before higher-level logic
- **Pros and Cons**
  - **Pros**: Easier test of low-level components, no UI needed
  - **Cons**: Requires drivers, may overlook high-level issues



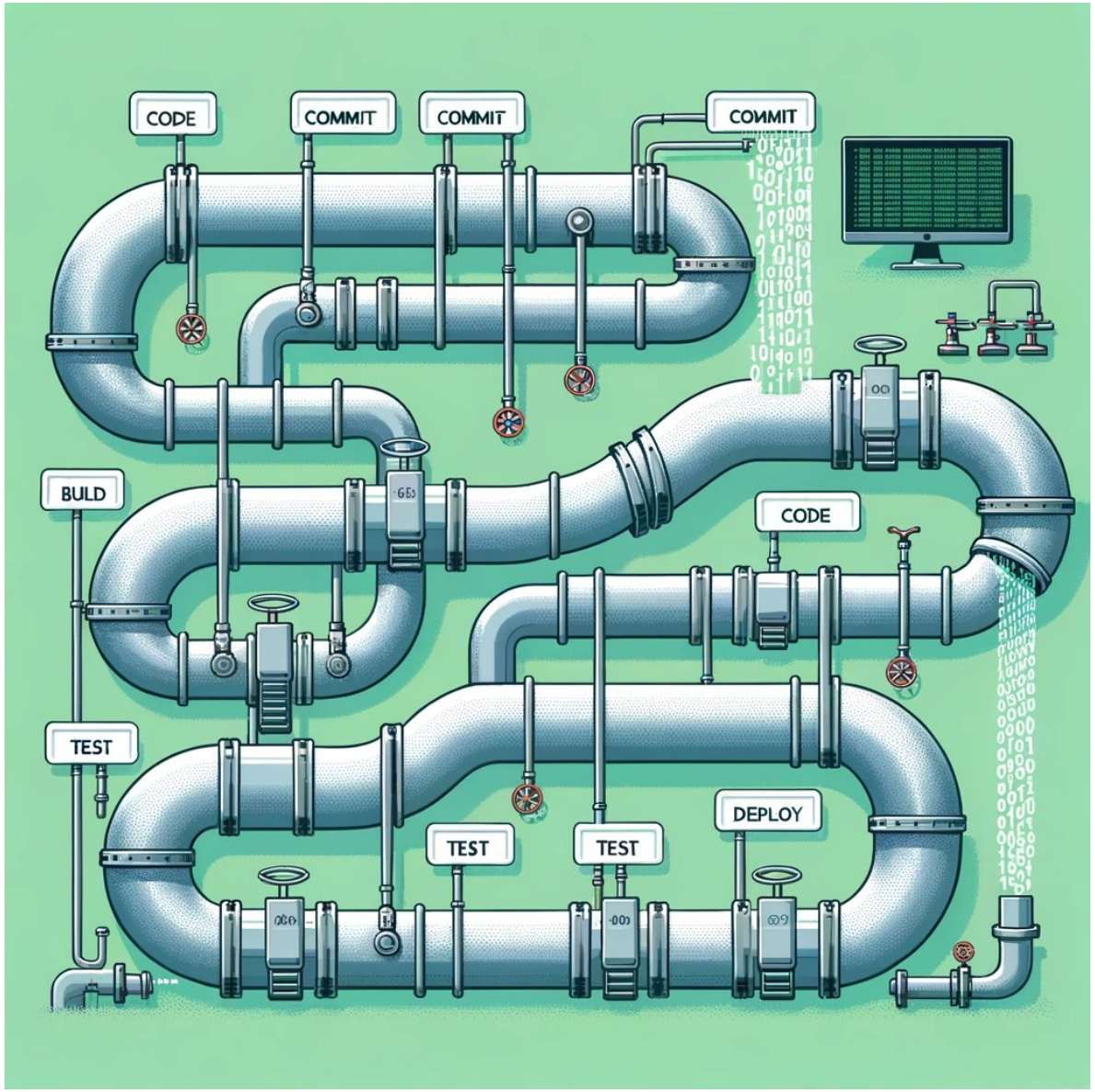
# SANDWICH (HYBRID)

- Combination of Top-down and Bottom-up
- **When to Use**: When different teams are working on different levels of the system
- **Pros and Cons**
  - **Pros**: Comprehensive, parallel development and testing
  - **Cons**: Requires both stubs and drivers, complex to manage



# CONTINUOUS INTEGRATION (CI)

- Ongoing process of integrating and testing after each code change
  - Continuous Integration (CI) is a software development practice where members of a team **integrate their work frequently**, usually multiple times a day. **Each integration is verified by an automated build and automated tests** to detect integration errors as quickly as possible.
- **When to Use**: Agile environments, projects requiring frequent changes
- **Pros and Cons**
  - **Pros**: Early bug detection, streamlined debugging
  - **Cons**: Requires robust automation, can be resource-intensive



# INTEGRATION TESTING TOOLS

- As you can imagine, there are lots of tools we can use to test the integration between multiple components
- You would be surprised how much we can achieve using JUnit – we can use it to perform both unit & integration testing

| Java Testing Frameworks | Continuous Integration Tools | API Testing Tools | Service Visualisation Tools | Performance Testing Tools |
|---|---|---|---|---|
| **JUnit:** We saw this last week, widely used for unit and integration tests | **Jenkins**: Supports building, deploying, and automating Java projects | **Postman**: For testing RESTful APIs in Java projects | **WireMock**: Stubs and mocks HTTP-based services | **Apache JMeter**: Design for load testing and can be used for functional tests |
| **TestNG:** Inspired by JUnit, extended capabilities, suitable for complex capabilities | **Travis CI**: Cloud-based and integrates well with GitHub projects | **SoapUI**: Suitable for testing SOAP-based web services | | **Gatling**: Scala-based but also used for integration testing in Java projects |

# DEMO: BOOK AND BOOKSTORE

- Let's take a look at a simple example of an integration test achieved using JUnit
- Book objects can be added to books ArrayList in Bookstore object via addBook method

**Book.java**

```java
public class Book {
    3 usages
    private String title;
    3 usages
    private double price;

    1 usage
    Book(String title, double price) {
        this.title = title;
        this.price = price;
    }

    // getters and setters omitted for brevity
}
```

**Bookstore.java**

```java
import java.util.ArrayList;

2 usages
public class Bookstore {
    3 usages
    private ArrayList<Book> books;

    1 usage
    public Bookstore() {
        this.books = new ArrayList<>();
    }

    1 usage
    public void addBook(Book book) {
        books.add(book);
    }

    1 usage
    public Book findBook(String title) {
        for (Book book : books) {
            if (book.getTitle().equals(title)) {
                return book;
            }
        }
        return null;
    }
}
```

```java
// import statements omitted for brevity
public class BookstoreTest {
    3 usages
    private Bookstore bookstore;

    @BeforeEach
    public void setUp() {
        bookstore = new Bookstore();
    }

    @Test
    @DisplayName("Test Book can be added and Found")
    public void testAddAndFindBook() {
        // Create a new book
        Book book = new Book( title: "To Kill a Mockingbird",  price: 10.0);

        // Add book to bookstore
        bookstore.addBook(book);

        // Find book in bookstore
        Book found = bookstore.findBook( title: "To Kill a Mockingbird");

        // Verify if the added book can be found
        assertNotNull(found,  message: "The book should be found");
        assertEquals(book, found,  message: "The found book should match the added book");
    }
}
```

You should hopefully remember these annotations from last week!

# EXAMPLES OF OTHER INTEGRATION TESTS FOR BOOKSTORE

- The previous slide showed a simple example of adding something to an ArrayList of another object and verifying it was indeed added by immediately searching for it
- The test method names to the right give some ideas of what else could be tested in a similar fashion

testAddSingleBookToEmptyBookstore
testAddMultipleBooksToBookstore
testAddAndRetrieveSingleBook
testAddAndRetrieveMultipleBooks
testAddAndRemoveSingleBook
testAddRemoveAndRetrieveBook
testAddAndUpdateBookDetails
testAddUpdateAndRetrieveBook
testAddAndClearBookstoreInventory
testAddMultipleAndRemoveSingleBook
testAddMultipleRemoveSingleAndRetrieveRemainingBooks

# INTEGRATION TESTS

- The integration test we just saw was testing the integration between **two classes**
- However, we often want to test the integration between our class(es)/method(s) and something external (e.g. a file, database, API)
- Let's take a look at an example which tests the integration between our code and an external **JSON** file
  - **JSON**: JavaScript Object Notation

# PRODUCTS.JSON

```
[
  {
    "id": "001",
    "name": "Laptop",
    "description": "A personal computer for mobile use.",
    "price": 999.99
  },
  {
    "id": "002",
    "name": "Smartphone",
    "description": "A portable device that combines mobile communication and computing functions.",
    "price": 599.99
  },
  {
    "id": "003",
    "name": "Smartwatch",
    "description": "A wearable computing device that closely resembles a wristwatch or other time-keeping device.",
    "price": 299.99
  }
]
```

# PRODUCT.JAVA

**Product.java**

```java
public class Product {
    private String id;
    private String name;
    private String description;
    private double price;

    public Product(String id, String name, String description, double price) {
        this.id = id;
        this.name = name;
        this.description = description;
        this.price = price;
    }
    // getters, and setters
}
```

# PRODUCTCATALOG.JAVA

**ProductCatalog.java (1 of 2)**

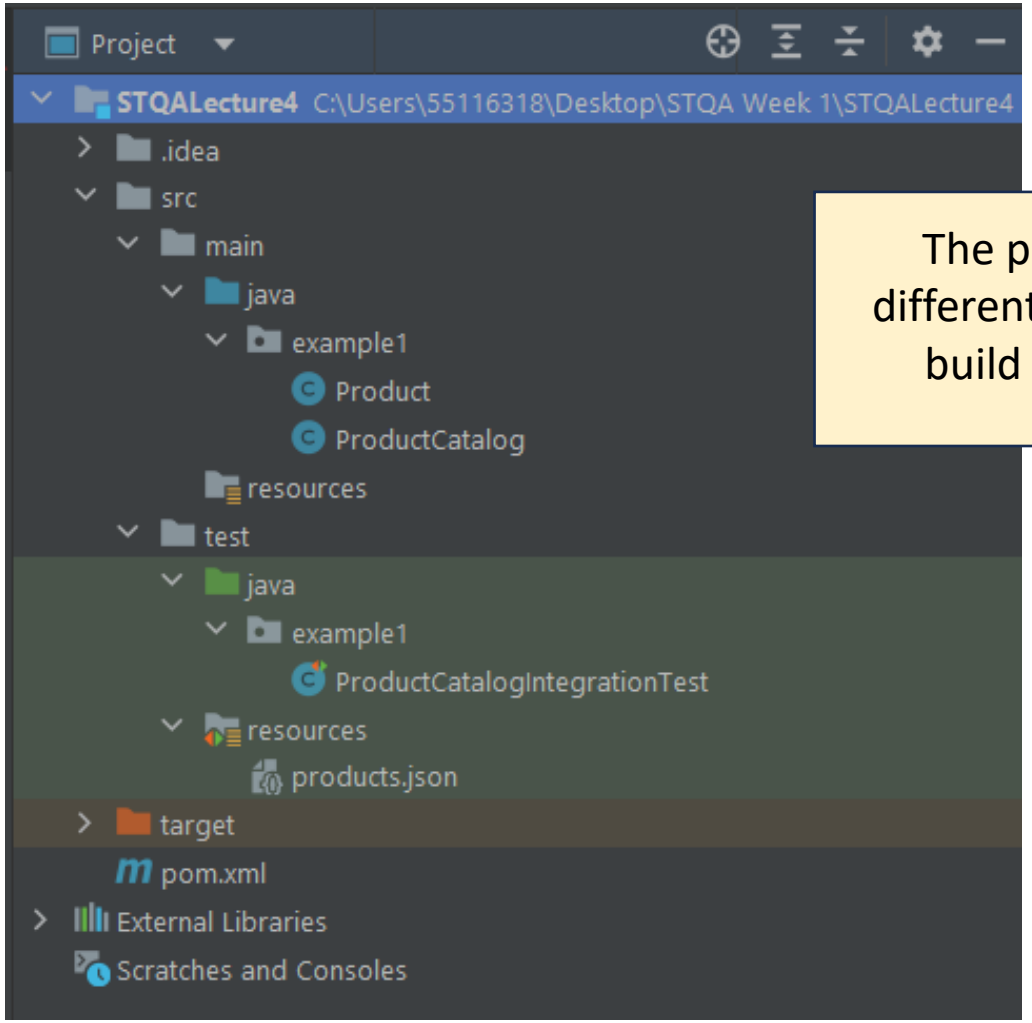
```java
public class ProductCatalog {
    private List<Product> products;
    private static final String CATALOG_FILE = "src/test/resources/products.json"; // Specify the correct path

    public ProductCatalog() throws IOException {
        this.products = loadProducts();
    }

    private List<Product> loadProducts() throws IOException {
        ObjectMapper mapper = new ObjectMapper();
        // Directly reads the JSON array into a List of Products
        return mapper.readValue(new File(CATALOG_FILE), new TypeReference<List<Product>>() {});
    }

    public void addProduct(Product product) throws IOException {
        // Adds a new product to the list
        products.add(product);
        // Writes the updated product list to JSON file
        saveProducts();
    }

    public void removeProduct(String productId) throws IOException {
        // Removes a product by ID from the list
        products.removeIf(p -> p.getId().equals(productId));
        // Writes the updated product list to JSON file
        saveProducts();
    }
```

**ProductCatalog.java (2 of 2)**

```java
    private void saveProducts() throws IOException {
        ObjectMapper mapper = new ObjectMapper();
        // Writes the current list of products back to the JSON file
        mapper.writeValue(Paths.get(CATALOG_FILE).toFile(), products);
    }

    public Product getProductById(String productId) {
        // Retrieves a product by ID
        return products.stream() Stream<Product>
                .filter(product -> product.getId().equals(productId))
                .findFirst() Optional<Product>
                .orElse( other: null);
    }

}
```

# PROJECT STRUCTURE

- This is what the project structure currently looks like




The project structure looks a little different here since I've used Maven (a build system we'll see in the labs)

# EXAMPLE INTEGRATION TEST – SETTING UP

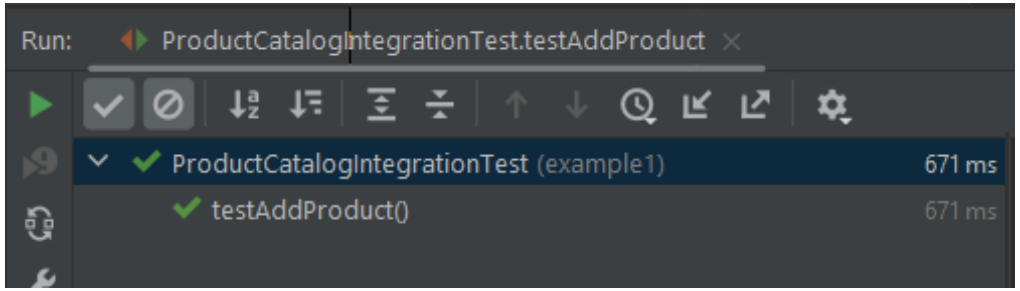
- **ProductCatalogIntegrationTest** test class contains an attribute of **ProductCatalog**
- A fresh object is created before each test due to the use of the **@BeforeEach** annotation

```
public class ProductCatalogIntegrationTest {

    private ProductCatalog catalog;

    @BeforeEach
    public void setUp() throws IOException {
        catalog = new ProductCatalog(); // loads the products from the JSON file
    }

}
```

# EXAMPLE INTEGRATION TEST – TESTING ADDPRODUCT

```java
public class ProductCatalogIntegrationTest {

    private ProductCatalog catalog;

    @BeforeEach
    public void setUp() throws IOException {
        catalog = new ProductCatalog(); // loads the products from the JSON file
    }

    @Test
    public void testAddProduct() throws IOException {
        Product newProduct = new Product( id: "004",  name: "Keyboard",  description: "A QWERTY keyboard.",  price: 12.99);
        catalog.addProduct(newProduct);
        Product retrievedProduct = catalog.getProductById( productId: "004");
        assertEquals(newProduct, retrievedProduct);
    }

}
```

Could also make this a ParamerterisedTest



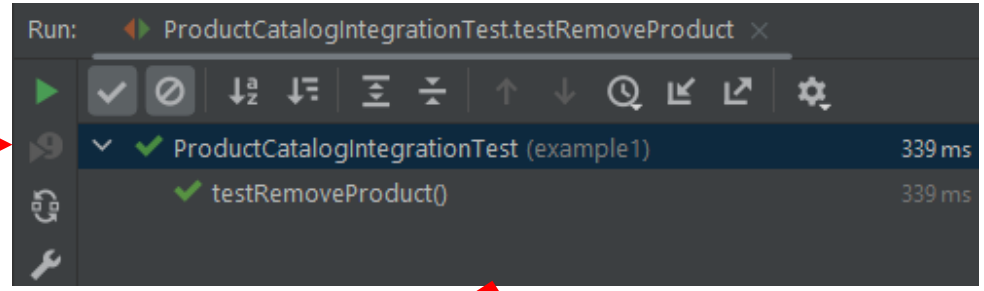
# PRODUCTS.JSON

As the Products.json file is fairly small, we could manually check it was added – but this would quickly become unfeasible as the file grows – and having to manually check this would make the automated test we just wrote kind of redundant

```
[
  {
    "id": "001",
    "name": "Laptop",
    "description": "A personal computer for mobile use.",
    "price": 999.99
  },
  {
    "id": "002",
    "name": "Smartphone",
    "description": "A portable device that combines mobile communication and computing functions.",
    "price": 599.99
  },
  {
    "id": "003",
    "name": "Smartwatch",
    "description": "A wearable computing device that closely resembles a wristwatch or other time-keeping device.",
    "price": 299.99
  },
  {
    "id": "004",
    "name": "Keyboard",
    "description": "A QWERTY keyboard.",
    "price": 12.99
  }
]
```

# EXAMPLE INTEGRATION TEST – TESTING REMOVEPRODUCT

```java
@Test
public void testRemoveProduct() throws IOException {
    catalog.removeProduct( productId: "001"); // Assuming product with ID "001" exists
    Product retrievedProduct = catalog.getProductById( productId: "001");
    assertNull(retrievedProduct);
}
```




```json
[
  {
    "id": "002",
    "name": "Smartphone",
    "description": "A portable device that combines mobile communication and computing functions.",
    "price": 599.99
  },
  {
    "id": "003",
    "name": "Smartwatch",
    "description": "A wearable computing device that closely resembles a wristwatch or other time-keeping device.",
    "price": 299.99
  },
  {
    "id": "004",
    "name": "Keyboard",
    "description": "A QWERTY keyboard.",
    "price": 12.99
  }
]
```

# SUMMARY

- We've had a quick introduction to Integration Testing
- Junit will be used again this week to perform Integration testing
- Different types depending on approach – e.g. big bang, bottom-up, top-down, CI

| Lab A – Intro to Integration Testing | Lab B – Intermediate Unit Testing |
| --- | --- |
| The first lab of the week will give us a gentle introduction to integration testing between different class **and** external data sources. | The second lab of the week will introduce some additional new content (similar to last week). |